1. Linux ⭐ ⭐ ⭐ — 2–4 YOE Scenario-Based Interview Q&AMOST IMPORTANT

1. Production server CPU suddenly reached 100%. How will you troubleshoot?
First, I check which process is consuming the CPU using tools such as top or ps.
Then I identify whether the process belongs to the application, database, Java process, or some system service.
I check application logs and recent deployments to see whether there was a code or configuration change.
If required, I restart the affected service only after understanding the impact, and then continue root-cause analysis.
Simple ga gurthupettuko:
CPU 100% ayithe first **ye process CPU consume chesthundo**, tarvatha **enduku consume chesthundo** investigate cheyyali.
Eng:
I identify the process consuming CPU, correlate it with application activity and recent changes, and investigate the root cause before taking corrective action.

---

2. Server memory usage is very high. What will you check?
First, I check memory and swap usage using free and top.
Then I identify which processes are consuming the most memory.
I check whether the application has a memory leak, whether too many processes are running, or whether the configured JVM or service memory is too high.
I also check whether the system is actively using swap because excessive swapping can impact performance.
Simple ga gurthupettuko:
Memory high ayithe **which process → memory usage → swap → application/JVM issue** check cheyyali.
Eng:
I check memory and swap usage, identify high-memory processes, and investigate application, JVM, or process-level memory consumption.

---

3. Disk space is 100% on a Production server. What will you do?
First, I confirm filesystem usage using df -h.
Then I identify which directory is consuming the space using commands such as du.
Common causes are application logs, old backups, temporary files, package caches, Docker data, or old application artifacts.
I don't blindly delete files. I identify what can safely be removed, clean it up, and then configure log rotation or retention to prevent recurrence.
Simple ga gurthupettuko:
Disk full ante **which filesystem → which directory → which files → safe cleanup → prevention**.
Eng:
I identify the filesystem and directories consuming space, safely clean unnecessary data, and implement retention or log rotation to prevent the issue from recurring.

---

4. df -h **shows free space, but the application says "No space left on device." What will you check?**
I would not assume that filesystem capacity is the only problem.
I check inode usage using df -i.
If inodes are exhausted, the server can fail to create new files even when disk capacity is available.
I also check for deleted files that are still held open by running processes.
Simple ga gurthupettuko:
df -h free unna kuda space error vasthe **inodes + deleted-but-open files** check cheyyali.
Eng:
I check inode exhaustion and open deleted files because a filesystem can have free capacity but still be unable to create files.

---

5. Server is responding very slowly, but CPU is normal. What will you investigate?
I check memory, swap usage, disk I/O, load average, network latency, and application-level performance.
A normal CPU doesn't mean the server is healthy.
For example, high disk I/O or excessive swapping can make the system slow even when CPU utilization is low.
Simple ga gurthupettuko:
CPU normal ante server healthy ani kaadu — **memory, swap, disk I/O, network, application** kuda check cheyyali.
Eng:
I investigate memory, swap, disk I/O, load, network latency, and application behavior rather than focusing only on CPU.

---

6. What is load average, and how do you interpret high load?
Load average indicates the number of tasks that are running or waiting for CPU or certain uninterruptible resources, commonly I/O.
I compare the load average with the number of CPU cores.
For example, a load of 8 on a 2-core system is more concerning than a load of 8 on a 16-core system.
I then investigate CPU utilization, I/O wait, and the processes contributing to the load.
Simple ga gurthupettuko:
Load average ni **CPU cores tho compare** chesi, tarvatha CPU/I/O wait reason identify cheyyali.
Eng:
I interpret load average relative to CPU capacity and then determine whether the load is caused by CPU contention, I/O wait, or other resource pressure.

---

7. A Linux service is down. How will you troubleshoot?
First, I check the service status using systemctl status.
Then I check service logs using journalctl or the application's own log files.

I verify configuration, ports, dependencies, permissions, disk space, and whether another process is already using the required port.
If there was a recent configuration or deployment change, I compare it with the last working state.
Simple ga gurthupettuko:
Service down → **status → logs → config → port → dependencies → recent changes**.
Eng:
I check the service status and logs first, then investigate configuration, dependencies, ports, permissions, resources, and recent changes.

---

8. A service starts and immediately stops. What will you check?
I first check the service status and logs.
Then I verify whether the application configuration is valid, required files exist, environment variables are available, ports are free, and dependencies are reachable.
I also check whether the process is exiting because of an application error or insufficient permissions.
Simple ga gurthupettuko:
Start ayyi immediate ga stop ayithe **logs + config + dependencies + permissions + port** check cheyyali.
Eng:
I inspect the service logs and verify configuration, dependencies, permissions, environment variables, and port availability to determine why the process exits.

---

9. A server rebooted unexpectedly. How will you investigate?
I check system logs around the reboot time.
I investigate whether it was caused by a kernel issue, hardware problem, out-of-memory condition, administrator action, cloud infrastructure event, or scheduled reboot.
I also correlate monitoring alerts and cloud-provider events if the server is running in AWS or another cloud platform.
Simple ga gurthupettuko:
Unexpected reboot ante **exact time identify chesi system logs + OOM + kernel + cloud events** check cheyyali.
Eng:
I identify the reboot time and correlate system logs, kernel events, resource exhaustion, administrative actions, and infrastructure events.

---

10. A process is consuming high CPU and you need to identify what application it belongs to. How?
I identify the PID using top or ps.
Then I inspect the process details using commands such as ps -fp <PID> and check the executable, command line, user, working directory, and open files if required.
This helps me map the process back to the application or service.
Simple ga gurthupettuko:
High CPU process ki **PID find → command/executable/user → application identify**.
Eng:
I identify the PID and inspect its command line, executable, user, and related process information to map it to the responsible application.

---

11. How do you troubleshoot a process that is continuously consuming memory?
First, I monitor the process over time instead of checking memory only once.
I verify whether memory usage continuously increases, which can indicate a memory leak.
I correlate the process with application logs, JVM metrics if applicable, recent deployments, and traffic patterns.
Then I involve the application team if the issue appears to be application-level.
Simple ga gurthupettuko:
Memory continuous ga increase ayithe **trend observe → leak suspect → logs/metrics → recent changes**.
Eng:
I monitor memory growth over time, correlate it with application behavior and deployments, and determine whether the issue is a memory leak or workload-related.

---

12. SSH to a server is failing. How will you troubleshoot?
First, I determine whether the server is reachable.
I check DNS resolution, network connectivity, port 22 connectivity, security-group or firewall rules, and whether the SSH service is running.
Then I check the SSH client error and server-side SSH logs.
If authentication is the issue, I verify the username, key, permissions, and authentication configuration.
Simple ga gurthupettuko:
SSH fail ante **network → port 22 → ssh service → firewall → authentication → logs**.
Eng:
I isolate SSH issues by checking connectivity, port access, the SSH service, firewall rules, authentication, permissions, and server-side logs.

---

13. SSH works for one user but fails for another. What will you check?
I compare the two users' configuration.
I check whether the failing user exists, whether the account is locked or expired, the user's shell, home directory permissions, SSH keys, .ssh permissions, and SSH authentication configuration.
I also check server-side authentication logs.

Simple ga gurthupettuko:
One user works, another fails ante **user account + shell + home + SSH key + permissions + logs** compare cheyyali.
Eng:
I compare the working and failing user accounts, including account status, shell, home directory, SSH keys, permissions, and authentication logs.

---

14. Application is running, but users cannot connect to it. What will you check?
I first verify that the process is running.
Then I check whether the application is listening on the expected port using ss or similar tools.
I verify whether it is listening on the correct interface, and then check host firewall, cloud security rules, load balancer configuration, and network connectivity.
Simple ga gurthupettuko:
Process running ante enough kaadu — **port listening + bind address + firewall + security rules + network** check cheyyali.
Eng:
I verify the process, listening port, bind address, firewall rules, security controls, and network path between the client and application.

---

15. Application is listening on localhost but not accessible remotely. What could be wrong?
The application may be bound only to 127.0.0.1.
In that case, it accepts connections only from the local machine.
I check the application's bind address configuration and change it appropriately, if required, to listen on the required interface.
I also verify firewall and security rules after changing the bind configuration.
Simple ga gurthupettuko:
127.0.0.1 meeda listen chesthe **local machine nunchi matrame access** avutundi.
Eng:
If the application is bound only to localhost, remote clients cannot access it, so I verify and correct the bind address and then check network controls.

---

16. How do you find which process is using a particular port?
I use commands such as ss -lntp or lsof -i to identify the process and PID associated with the port.
For example, if port 8080 is already occupied, I identify which process owns it before starting another application.
Simple ga gurthupettuko:
Port already use lo unda? **Port → PID → process → application** find cheyyali.
Eng:
I use ss or lsof to identify the process and PID listening on the required port.

---

17. Application deployment fails because the target directory has permission denied. What will you check?
I check the directory owner, group, permissions, and the user under which the deployment process is running.
I avoid simply using chmod 777.
Instead, I provide the minimum required permissions through correct ownership, groups, ACLs, or service configuration.
Simple ga gurthupettuko:
Permission issue lo 777 ivvakunda **owner + group + required permission** correct cheyyali.
Eng:
I verify the deployment user and directory ownership and provide only the minimum permissions required instead of using overly permissive settings.

---

18. What is the difference between a process and a service from a troubleshooting perspective?
A process is a running instance of a program.
A service is typically a managed application or daemon controlled by an init/service manager such as systemd.
For troubleshooting, I may identify the process first and then check the corresponding service configuration, status, and logs.
Simple ga gurthupettuko:
Process = **running program**; Service = **managed application/daemon**.
Eng:
A process is a running program instance, while a service is typically a managed daemon or application controlled by the system service manager.

---

19. A server has many zombie processes. What will you investigate?
Zombie processes are processes that have completed execution but whose parent has not collected their exit status.
I identify the zombie processes and their parent PIDs.
Then I investigate the parent application because the real issue is usually related to how the parent handles child processes.
Simple ga gurthupettuko:
Zombie process ni direct ga kill cheyyadam kanna **parent process identify chesi parent-side issue** investigate cheyyali.
Eng:
I identify zombie processes and their parent processes, then investigate why the parent is not properly reaping child processes.

---

20. A process is stuck and does not terminate normally. What will you do?
I first try a graceful termination using SIGTERM.
If the process does not respond and the situation requires immediate action, I may use SIGKILL, but only after understanding the impact.
Before force-killing a Production process, I verify whether it could cause data loss or service disruption.

Simple ga gurthupettuko:
First **graceful stop**, force kill only if required and impact understood.
Eng:
I prefer graceful termination first and use forceful termination only when necessary after assessing data and service impact.

⭐ ⭐ **IMPORTANT**
21. How do you troubleshoot high disk I/O?
I check disk utilization and identify which processes are generating heavy I/O.
I investigate application logs, database activity, large file operations, backups, and temporary files.
I also check whether the underlying storage is experiencing latency.
Simple ga gurthupettuko:
Disk slow ante **which process → I/O usage → application/database/storage latency**.
Eng:
I identify the processes generating I/O and investigate application activity, storage latency, backups, and other heavy disk operations.

---

22. How do you identify large files consuming disk space?
I first identify the filesystem using df -h.
Then I use du to find large directories and files and can use find to locate files above a particular size.
I check whether those files are safe to delete before removing anything.
Simple ga gurthupettuko:
Disk full → **filesystem → directory → large files → safe cleanup**.
Eng:
I identify the affected filesystem, locate large directories and files, verify their purpose, and safely clean unnecessary data.

---

23. Log files are growing rapidly. What will you do?
First, I identify which log is growing and why.
I check application errors, traffic spikes, debug logging, and recent configuration changes.
For immediate disk pressure, I safely manage the logs according to the organization's retention process and ensure log rotation is configured.
Simple ga gurthupettuko:
Logs fast ga grow ayithe **which log + why growing + disk impact + log rotation**.
Eng:
I identify the source of excessive logging, address the underlying cause, manage disk usage safely, and verify log rotation and retention.

---

24. What happens if /var **becomes full?**
Applications and system services may start failing because /var commonly contains logs, caches, spool data, and other runtime information.
I identify what is consuming /var, clean only safe unnecessary data, and check log rotation and retention.
Simple ga gurthupettuko:
/var full ayithe **logs/cache/runtime data** వల్ల services affect avvachu.
Eng:
If /var is full, I identify the specific consumer, safely clean unnecessary data, and correct retention or rotation policies.

---

25. A server cannot resolve a domain name, but it has network connectivity. What will you check?
I separate network connectivity from DNS resolution.
I check /etc/resolv.conf, configured DNS servers, DNS reachability, and use tools such as dig or nslookup.
I also verify whether the issue affects one server or multiple servers.
Simple ga gurthupettuko:
Internet/network works but domain resolve kaadu ante **DNS configuration + DNS server reachability** check cheyyali.
Eng:
I verify DNS configuration, resolver reachability, DNS responses, and whether the problem is isolated to one server or affects the wider environment.

---

26. A server can ping another server, but the application port is unreachable. Why?
Ping tests ICMP connectivity; it does not prove that the application port is accessible.
I check whether the target application is listening, whether the port is blocked by firewall/security rules, and whether network ACLs or other controls are involved.
Simple ga gurthupettuko:
Ping success ≠ application port success.
Eng:
Ping only verifies ICMP connectivity, so I separately verify the application's listening port, firewall rules, security controls, and network path.

---

27. How do you troubleshoot a sudden network connectivity issue between two Linux servers?
I test connectivity in layers.
First, I check DNS if hostnames are used. Then I test basic connectivity, port connectivity, routing, firewall rules, and the application itself.
I compare the working and failing servers and check for recent infrastructure or security-rule changes.

Simple ga gurthupettuko:
Network issue ni **DNS → connectivity → port → route → firewall → application** layers lo isolate cheyyali.
Eng:
I troubleshoot network connectivity layer by layer, from DNS and routing through port access, firewall rules, and application availability.

---

28. How do you check whether a Linux server has enough CPU and memory capacity for a new application?
I review current CPU, memory, load, disk, and network utilization.
I also look at historical monitoring data rather than relying only on current usage.
Then I estimate the application's expected resource requirements and determine whether the server has sufficient headroom.
Simple ga gurthupettuko:
New app deploy mundu **current usage + historical usage + expected requirement + headroom** check cheyyali.
Eng:
I evaluate current and historical resource utilization and compare it with the application's expected resource requirements and available capacity.

---

29. A deployment works when executed manually but fails when executed through Jenkins on the Linux server. What will you check?
I compare the manual and Jenkins execution environments.
I check the Linux user, environment variables, PATH, working directory, permissions, SSH keys, credentials, and installed tools.
A common reason is that Jenkins runs under a different user with a different environment.
Simple ga gurthupettuko:
Manual success, Jenkins fail ante **same user/environment ani assume cheyyakudadhu**.
Eng:
I compare the Jenkins and manual execution contexts, especially user, PATH, environment variables, permissions, working directory, and credentials.

---

30. A shell script works manually but fails from Jenkins. Why?
I first compare the execution user and environment.
The script may depend on a PATH entry, environment variable, current directory, shell configuration, or permission that exists in the interactive session but not in Jenkins.
I make the script explicit and avoid depending on interactive shell configuration.
Simple ga gurthupettuko:
Manual shell environment ≠ Jenkins environment.
Eng:
I compare the execution environments and remove dependencies on interactive shell settings, implicit PATH values, or user-specific configuration.

---

31. How do you safely restart a Production Linux service?
First, I understand the impact and confirm whether redundancy or failover is available.
I check the current service health and active connections if relevant.
Then I restart the service during the appropriate change window or according to the incident procedure and immediately verify service status, logs, port availability, and application health.
Simple ga gurthupettuko:
Production restart ante **impact → redundancy → restart → verify health**.
Eng:
I assess the impact and availability requirements first, restart through the approved process, and immediately validate service and application health.

---

32. How do you find when a particular file was modified?
I use stat to inspect file timestamps.
I distinguish between modification time, access time, and metadata/status changes depending on what I need to investigate.
For incident troubleshooting, I correlate the file timestamp with deployment and application logs.
Simple ga gurthupettuko:
File change time investigate cheyyali ante stat **+ deployment/application logs correlation**.
Eng:
I use file metadata such as stat and correlate timestamps with deployment and application logs to identify when changes occurred.

---

33. A file was accidentally deleted from a Production server. What will you do?
I first determine whether the file is still held open by a running process.
If so, recovery may be possible from the process's open file descriptor.
Otherwise, I check backups, snapshots, version-controlled configuration, or other approved recovery mechanisms.
I avoid making random changes that could overwrite recoverable data.
Simple ga gurthupettuko:
Deleted file ante **open process → backup/snapshot → approved recovery** check cheyyali.
Eng:
I check for open file descriptors and then use approved backups, snapshots, or recovery mechanisms rather than making uncontrolled changes.

---

34. How do you troubleshoot a server where a scheduled cron job is not running?
I check whether the cron entry exists and whether the cron service is running.
Then I check the cron logs and execute the command manually using the same user.
I also verify absolute paths, environment variables, permissions, working directories, and output/error redirection.
Simple ga gurthupettuko:
Cron fail ante **entry → cron service → logs → user → PATH/env → permissions → command**.
Eng:
I verify the cron configuration and service, inspect logs, and test the command with the same user and environment used by cron.

---

35. A cron job runs manually but fails from cron. Why?
Cron does not necessarily have the same environment as an interactive shell.
The command may depend on PATH, environment variables, working directory, or shell configuration.
I use absolute paths, explicitly configure required environment variables, and capture stdout/stderr for troubleshooting.
Simple ga gurthupettuko:
Manual success but cron fail = **environment difference** most common suspect.
Eng:
I compare the interactive and cron environments and make paths, environment variables, working directories, and logging explicit.

🔥 **BONUS — Interview Follow-up Scenarios**
36. Server load is high, CPU is low, and I/O wait is high. What does that tell you?
High I/O wait suggests that processes are spending significant time waiting for storage operations rather than actively using CPU.
I investigate disk latency, heavy disk consumers, database activity, filesystem issues, and underlying storage performance.
Simple ga gurthupettuko:
High load + low CPU + high I/O wait → **disk/storage bottleneck suspect cheyyali**.
Eng:
High I/O wait with low CPU utilization indicates that processes may be blocked waiting for storage operations, so I investigate disk and storage performance.

---

37. A Production server is running out of memory every few days and requires a restart. How would you approach it?
I would not treat restarting as the permanent solution.
I monitor memory usage over time and identify which process grows continuously.
For Java applications, I check JVM heap and garbage-collection behavior. For other applications, I inspect process memory trends and application logs.
Then I work with the application team to address the underlying memory issue.
Simple ga gurthupettuko:
Every few days restart cheyyadam solution kaadu — **memory growth trend + culprit process + root cause** find cheyyali.
Eng:
I treat recurring memory exhaustion as a root-cause problem, identify the process showing abnormal growth, and investigate application or runtime memory behavior.

---

38. Production server suddenly became unreachable. What is your first approach?
I first determine whether the problem is limited to SSH or whether the server itself is unreachable.
I check monitoring, network connectivity, DNS, cloud instance status, security rules, and whether other services are affected.
If console or out-of-band access is available, I use it to inspect CPU, memory, disk, network, and system logs.
Simple ga gurthupettuko:
Server unreachable ante immediate ga SSH ni blame cheyyakunda **server health + network + cloud + services** isolate cheyyali.
Eng:
I first determine whether the issue is network access, host availability, or service-specific, and then use monitoring and console access to isolate the failure.

---

39. A new deployment causes the Linux server CPU to spike. What will you do?
I correlate the CPU spike with the deployment timestamp.
I identify the process responsible and compare its behavior with the previous application version.
I check application logs, traffic, configuration changes, thread usage, and resource limits.
If Production impact is significant, I prioritize service restoration through rollback or another approved recovery method.
Simple ga gurthupettuko:
Deployment tarvatha CPU spike → **timeline correlate → process identify → old version compare → impact unte rollback/recovery**.
Eng:I correlate the resource spike with the deployment, identify the responsible process, compare it with the previous version, and use rollback or another safe recovery method if required.

---

40. Interviewer: "You have only 5 minutes to troubleshoot a Production Linux issue. How do you approach it?"
I follow a structured approach rather than randomly running commands.
First, I understand the user impact and scope. Then I check system health—CPU, memory, disk, load, network, and service status.
Next, I check recent changes and relevant logs.
I identify the most likely failure point, take the safest recovery action if required, and then continue with root-cause analysis after service stability is restored.
Simple ga gurthupettuko:
5 minutes incident ante **Impact → Health → Recent changes → Logs → Isolate → Safe recovery → RCA**.
Eng:I prioritize impact assessment, rapid health checks, recent changes, logs, failure isolation, and safe service restoration, followed by detailed root-cause analysis.